STAT 418 · FINAL PROJECT · JUNE 2026

U.S. Retail Gasoline Price Forecast

A deployed data science application that collects weekly EIA gasoline prices, trains two forecasting models, serves predictions through Flask, and presents them in an interactive Streamlit dashboard.

1,729

Weekly Observations

1993–2026 EIA data

2

Models Served

XGBoost & SARIMA

2

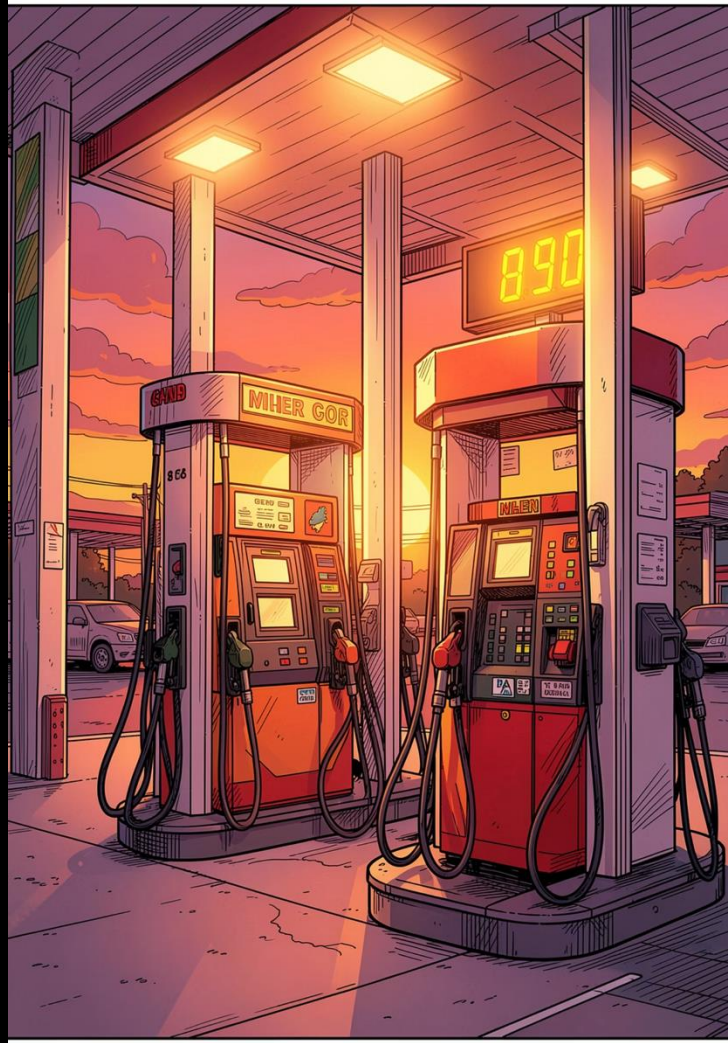
Live Services

Flask API + Streamlit UI

Mike Wu

Streamlit app: https://fuel-price-streamlit-oluqiqxkmq-uw.a.run.app

Flask API: https://fuel-price-api-oluqiqxkmq-uw.a.run.app



MOTIVATION

Fuel Prices Are Volatile Enough to Make Short-Term Forecasting Useful

$0.949

Lowest Historical

per gallon

$5.107

Highest Historical

per gallon

$4.621

Latest Observation

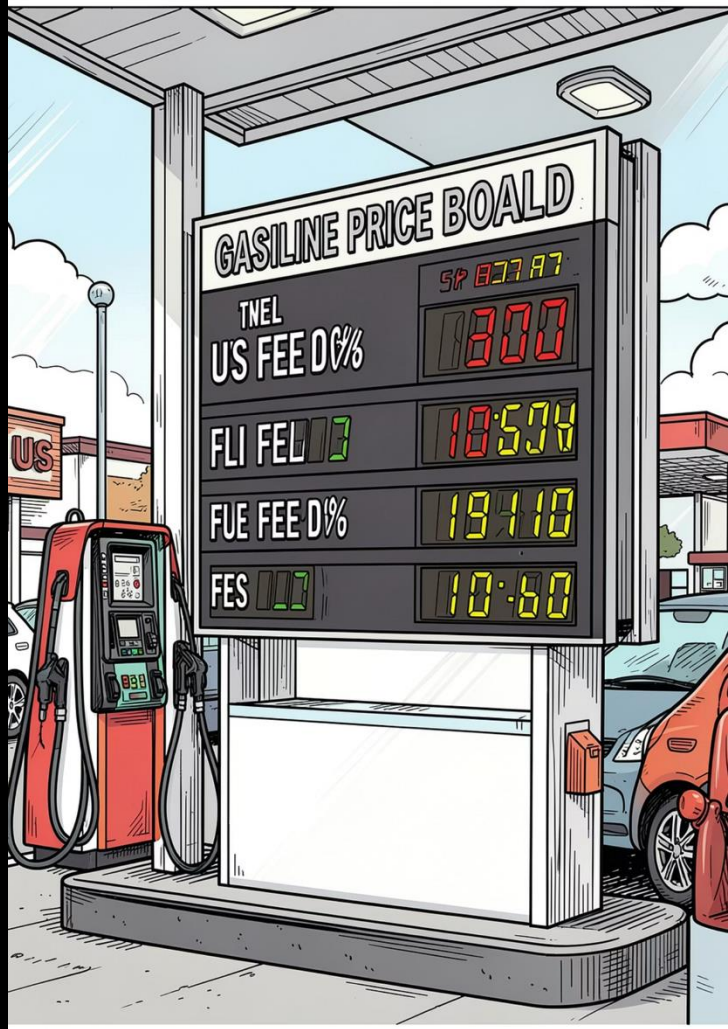
May 18, 2026

User Problem

Price movement is hard to reason about from raw weekly tables. A useful class demo needs both an interface and a deployed model endpoint.

Project Objective

Make model choice and forecast horizon interactive. Deploy the API and app separately so the architecture matches production patterns.



DATA PIPELINE

EIA Weekly Gasoline Prices Become Model-Ready Lag Features

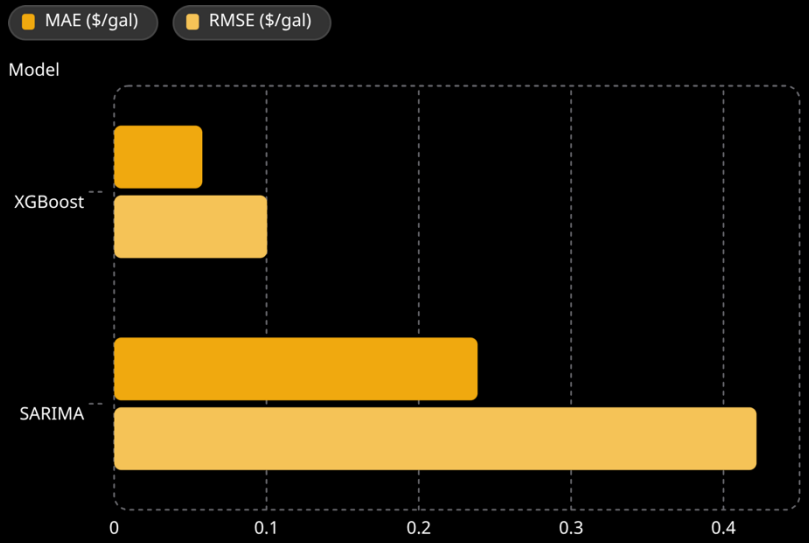
Explainable features: recent lags capture price level and momentum; calendar fields capture weekly seasonality.

Feature Engineering

- Lags 1, 2, 4, 8, 52: recent level, momentum, annual seasonality
- Month + ISO week: calendar structure
- 1,729 rows: Apr 5, 1993 to May 18, 2026

MODEL EVALUATION

XGBoost Won the 52-Week Holdout Comparison



XGBoost — Default Model

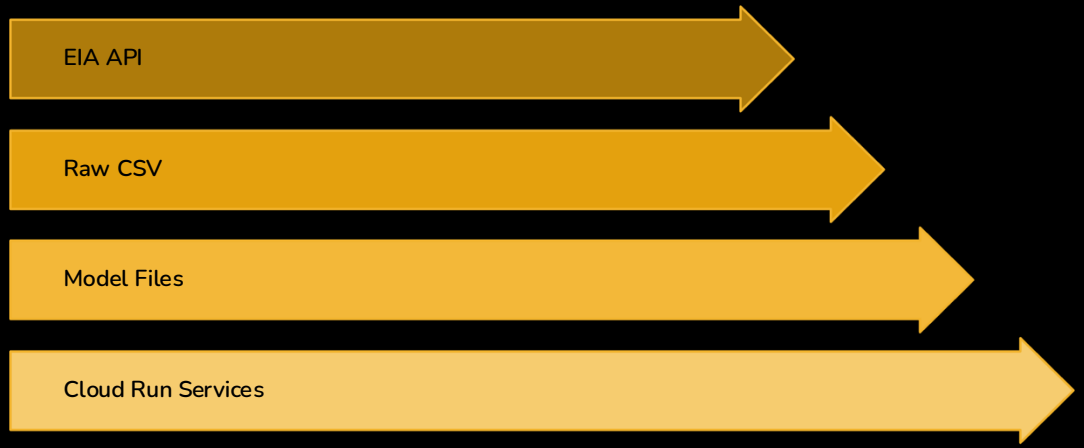
Recursive multi-step forecast from lag features plus month and week-of-year. Recent lags explain more of the short-term signal than a univariate seasonal structure alone.

SARIMA — Baseline

Seasonal weekly time-series baseline: SARIMA(1,1,1)(1,1,1,52). Both models are available in the app, but XGBoost's lower holdout error makes it the default.

ARCHITECTURE

The App and Model API Are Deployed as Separate Cloud Run Services



EIA data flows through a raw CSV snapshot into trained model artifacts, served by a Flask API and consumed by a separate Streamlit UI — both running on Cloud Run.

Why Separate Services?

- UI and API deploy independently
- API testable with curl/Postman
- Artifact Registry stores images
- API memory: 1 Gi for SARIMA

Endpoints

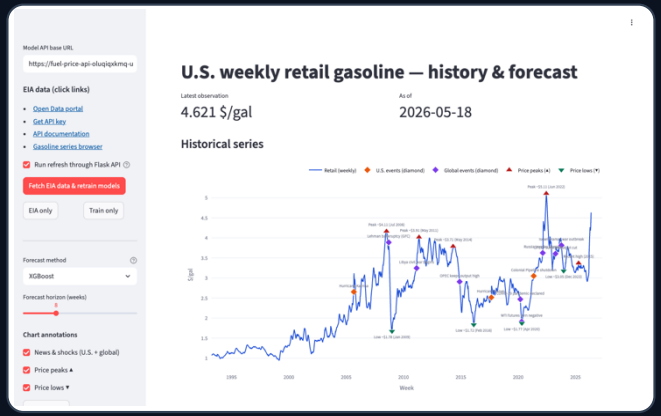
/health
Liveness check

/metadata
Model info

/predict
Forecast JSON

LIVE APP SCREENSHOTS

Click to reveal: Streamlit UI, then Flask API JSON



1 Streamlit dashboard on Cloud Run

```
POST /predict
{
"method": "XGBoost (recursive lags)",
"first_forecast": {
"period": "2026-05-25",
"forecast_price": 4.576
},
"last_forecast": {
"period": "2026-07-13",
"forecast_price": 4.404
}
}
```

2 Flask API returns the forecast payload

LIVE DEMO

The Demo Shows Streamlit Calling Flask in Real Time

01

Choose Method

Select XGBoost or SARIMA, then set the forecast horizon.

02

Submit Forecast

Streamlit sends horizon + method to Flask /predict.

03

API Forecast

`Flask loads the trained model and returns weekly JSON forecasts.`

04

View Results

The app displays the forecast chart/table while the raw API call proves the backend works independently.

Live demo links: Streamlit app + POST https://fuel-price-api-oluqiqxkmq-uw.a.run.app/predict

REFLECTION

The Main Work Was Turning a Model Into a Reliable Deployed System

Challenges Solved

- Cloud Run needed separate API/UI services and correct MODEL_API_URL wiring
- SARIMA artifact was too large; compact parameter storage fixed deployment
- SARIMA needed 1 Gi Cloud Run memory to avoid instance termination

AI Assistant Role

- Scaffolded Flask/Streamlit integration and Docker commands
- Generated draft code that I checked against course requirements
- Lesson: pair AI help with tests, logs, and explicit verification

Future Work: scheduled EIA refresh, confidence intervals, SHAP feature importance, and leaner dependency files for faster rebuilds.